

TriMeter

A GPS-based tricycle fare meter for Cabanatuan City

Mobile Application Technology: Final Case Study

BSIT-4C

Group members

John Mathew Ramoso

Neo Arwen De Guzman

Charles Benedict Leodones

Project files: https://github.com/mintchoco0624/trimeter-app

1. Project Information

Title: TriMeter: a GPS-based tricycle fare meter for Cabanatuan City. **Platform:** Android (Ionic 9, Angular 22 standalone components, Capacitor 8). **Members:** John Mathew Ramoso, Neo Arwen De Guzman, Charles Benedict Leodones (BSIT-4C).

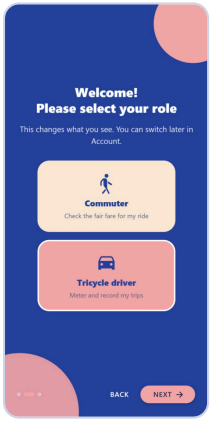
2. Problem and Proposed Solution

Problem. Cabanatuan City is known as the "Tricycle Capital of the Philippines", and thousands of commuters ride tricycles every day. The city has an official fare matrix, but fares in practice are inconsistent: TODAs apply their own rates, drivers estimate by feel, and studies describe overcharging as rampant, with drivers charging up to 300% more than the prescribed rates. Passengers, especially students, seniors, PWDs and visitors, have no easy way to check the fair fare.

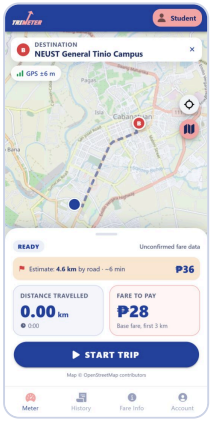
Intended users. Primary: commuters (students, workers, seniors, PWDs, tourists). Secondary: tricycle drivers, who gain a neutral reference that prevents disputes.

Proposed solution. A mobile app that measures the real distance of a ride with the phone's GPS and applies the city fare matrix live, then gives a receipt with a fare breakdown and an overcharge check. The group's original proposal also included accounts, ID-verified discounts, multiple languages, real-time gas prices, driver QR pairing and an LGU dashboard. These were set aside as future improvements to keep the scope realistic for a two-week project.

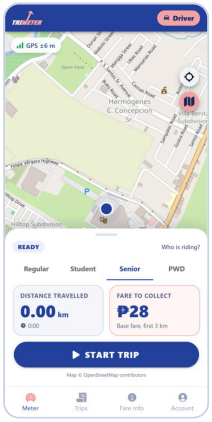
3. Application Features



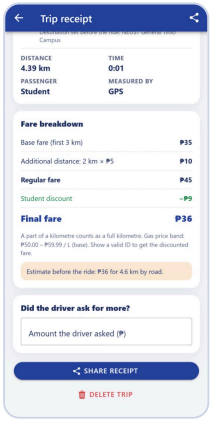
Role selection



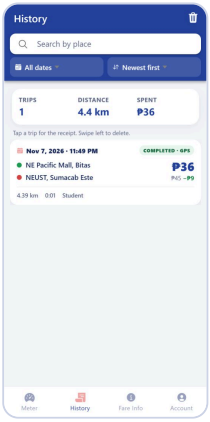
Commuter meter with fare preview



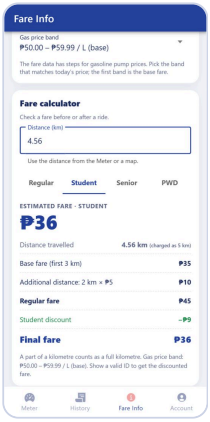
Driver meter



Receipt and overcharge check



History with search and filters



Fare calculator

- **Onboarding and roles:** the user chooses *Commuter* or *Tricycle driver* (as in the proposal prototype). Roles are separate from the passenger type (Regular, Student, Senior, PWD), which sets the fare.
- **Live meter (automatic mode):** a commuter can search "Where to?" and see the road distance and expected fare before riding; a driver picks the passenger type for each trip. After *Start Trip*, the map draws the route and the distance, time and fare update live, with a GPS signal indicator, an offline indicator and the trip status. *End Trip* asks for confirmation; a trip under 50 m cannot be saved as a paid trip.
- **Trip receipt:** route map, places, distance, time, passenger type and a fare breakdown (base fare, additional distance, regular fare, discount, final fare). Commuters can type what the driver asked to see any overcharge in pesos and percent. The receipt can be shared.

- **Manual estimate (backup mode):** pick A and B by search or on the map to get the road distance and fare. The app offers it whenever GPS is unavailable.
- **History:** trip cards with date, places, distance, duration, passenger type, regular fare, discount, final fare and status; search by place, date filter, sorting, totals, and delete with confirmation.
- **Fare info and Account:** the fare matrix (ordinance no., effective date, source, status), a fare calculator, example fares, 8 FAQs and complaint steps. Account holds the local profile, role switch, discount and privacy information, and log out.

4. Technical Implementation

The project was created with the standard Ionic workflow: `ionic start trimeter-app blank --type=angular-standalone --capacitor`, then every feature was generated with `ionic generate`, so each has its own TypeScript, HTML, SCSS and test file. The app has 8 pages, 4 reusable components (map, fare breakdown, place search, fare calculator), 9 services, 1 route guard, 2 pipes and 3 model files. Pages only display data; the work is done in services. The Android project was added with `ionic capacitor add android`.

Capacitor capabilities

- **Geolocation** (`services/location.ts`): `watchPosition` with high accuracy (about one fix every 2 s on Android). Each fix passes `judgeFix()`, which rejects fixes less accurate than 35 m, movements under 5 m (jitter while waiting) and jumps faster than 30 m/s. Accepted fixes extend the route, and the distance is summed with the haversine formula. Results are Angular signals, so the UI updates automatically. The Screen Wake Lock API keeps the screen on during a trip. Before the first permission prompt the app explains why it needs location, and it tracks only between *Start Trip* and *End Trip*.
- **Haptics** (`services/feedback.ts`): a light tap when the fare increases, and stronger taps at the start and end of a trip.
- **Share**: the receipt is shared as plain text, or copied to the clipboard where sharing is unavailable.
- **Map:** Leaflet with OpenStreetMap tiles (`components/trip-map`).

Fare computation

`services/fare.ts` is a pure function. The base fare covers the first 3 km, and each succeeding kilometre adds the per-km rate, with a part of a km charged as a full km (`Math.ceil`). The distance is rounded to 10 m first, so GPS noise (e.g. 3.004 km) is not billed as an extra km. Students, seniors and PWDs use the discounted column. Bundled rates follow City Ordinance No. 050-2026 as compiled by the community BetterCabanatuan project: ₱35 / ₱28 for the first 3 km and ₱5 / ₱4 per succeeding km, with gasoline-price bands.

Data handling

- **HttpClient GET** (`services/fare-matrix.ts`): uses the `inject(HttpClient)` / `subscribe({ next, error })` pattern from class to load the fare matrix JSON. If the online copy fails, the app uses the copy saved in Preferences, then the copy bundled in `assets/`. Fare changes can therefore reach users without a new APK. GET requests also call OpenStreetMap Nominatim (place names) and OSRM (road distance). All endpoints are public and need no API keys.
- **Local data** (`services/trip-storage.ts`): Capacitor Preferences stores the trip history as JSON with create (`add`), read (`load` / `get`), update (`update`: what the driver asked) and delete (`remove` / `clear`). The local profile (role, name, passenger type, gas band) is stored the same way; there is no server account. History search, date filter and sorting are a pure function (`services/trip-query.ts`) so they are unit-tested.

- **Validation and errors:** the calculator checks the typed distance; a destination closer than 100 m is rejected; failed saves offer a retry; offline, place search explains the problem and road distance falls back to a labelled straight-line estimate.
- **Navigation:** a functional route guard sends first-time users to onboarding; the tabs are Meter, History (drivers: Trips), Fare Info and Account. Android permissions: fine and coarse location, internet, vibrate.

5. Testing and Evaluation

Automated tests: **73 unit tests** (Vitest) for the fare rules, receipt breakdown, GPS filter, history search, calculator validation, settings, roles and pages, and **8 end-to-end tests** (Playwright, Chrome). The end-to-end tests replay a real 4.56 km road route (NE Pacific Mall → NEUST, from OSRM) as GPS fixes, with the online services mocked so results are repeatable. All automated tests pass, and `ng lint` is clean.

Feature / task tested	Expected result	Actual result	Status
Role selection (commuter / driver)	Role saved; tabs and Meter change	As expected	**Pass**
GPS meter, 4.56 km route, regular	₱45 (₱35 + 2 km × ₱5)	₱45	**Pass**
Driver, student passenger, same route	₱36; receipt shows ₱45 − ₱9 discount	₱36, as expected	**Pass**
Destination preview (commuter)	4.6 km by road, ₱45 before the ride	As expected	**Pass**
GPS jitter while standing still (30 fixes)	0.00 km added	0.00 km	**Pass**
End a trip with no distance	Not saved; "Discard trip" offered	As expected	**Pass**
Save, reload, search and delete a trip	Kept after reload; search filters; delete confirmed	As expected	**Pass**
Overcharge check (₱90 asked vs ₱45)	"Overcharged by ₱45 (100%)"	As expected	**Pass**
Calculator: 4.56 km student; invalid input	₱36; error message for 0 or text	As expected	**Pass**
Offline place search / road distance	Clear message; straight-line fallback	As expected (unit test)	**Pass**
Real ride on an Android phone (APK)	Distance and fare update live	[fill in after the device test]	**To verify**

Issues found and limitations

- During development: the map covered the page header and the bottom panel sat behind the tab bar; the map credit was hidden; a route guard called `inject()` after `await`. All were found by testing and fixed.
- Phone GPS drifts near tall buildings and in bad weather, so the distance can differ slightly from the road distance.
- The app must stay open during a trip; Android can pause GPS updates in the background.
- The fare matrix is a community copy of Ordinance No. 050-2026. A Rappler fact check (April 2026) reported it was still a draft, so the app labels it "Unconfirmed" and shows the effective date as "Not confirmed". An unverified complaint hotline was removed.

- Roles and the profile are stored only on the phone: there is no real login, and discounts depend on the driver checking a valid ID.
- Maps, place names and manual road distance need an internet connection (the GPS meter itself does not).

Recommended improvements (from heuristic review)

- A foreground service so tracking continues with the screen off (*visibility of system status*, *error prevention*).
- Verify the fare matrix and a complaint hotline with the Cabanatuan City Government.
- A Filipino language option and a larger-text mode; some secondary labels are 11–12 px (*match with the real world*, accessibility).
- Later: driver QR pairing, ID-verified discounts and an LGU reporting dashboard from the original proposal.

6. AI Usage Disclosure

Tool: Claude Code (Anthropic), an AI coding assistant used in VS Code. **How it assisted:** scaffolding the project with the Ionic CLI, writing and refactoring the services, pages and tests, implementing the group's improvement brief (roles, fare calculator, history filters, error handling), researching and fact-checking the fare data sources, and producing the Remotion presentation video and script. **What the group did:** reviewed the generated code, ran the unit and end-to-end tests, tested the APK on an Android phone, and fixed the issues found. The group can explain the main code (`location.ts`, `fare.ts`, `fare-matrix.ts`, `trip-storage.ts`). No passwords, private keys or personal data were used in prompts or code.

> Before submitting, the group should confirm the device test and edit this section to describe exactly what each member reviewed and changed.

References

- BetterCabanatuan. *bettercabanatuan-fare* (community fare data, Ordinance No. 050-2026). https://github.com/BetterCabanatuan/bettercabanatuan-fare
- Rappler (2026). Fact check: New Cabanatuan City fare matrix still a draft, ordinance not yet in force. https://www.rappler.com/newsbreak/fact-check/new-cabanatuan-city-fare-matrix-ordinance-draft-not-yet-in-force/
- ResearchPublish (2016). Cabanatuan City: Tricycle Capital of the Philippines. https://www.researchpublish.com/upload/book/Cabanatuan%20City-3270.pdf
- Economic Suitability of Electronic Fare Meters for Tricycle Drivers in Cabanatuan City (2025). ResearchGate.
- Ionic Framework, Capacitor and Angular documentation; OpenStreetMap, Nominatim and OSRM.